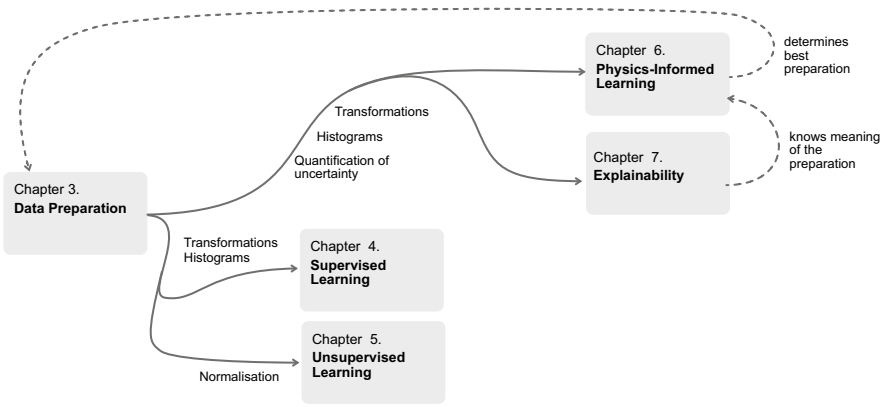


**Fig. 3.1** Overview of the relationship of Chap. 3 with the following chapters

## 3.1 Goals of Preprocessing

Data preprocessing encompasses all necessary steps that generate a meaningful dataset from a set of raw data. Meaningful are all processes that maintain and, if possible, increase the interpretability of the data.

Data preprocessing includes the following steps, which we want to discuss in more detail in this chapter:

- **Data cleaning.** Here, the dataset is examined for problems and prepared. In practice, these problems occur more frequently and this step is often labor-intensive. We can distinguish the following sub-points:
  - Find and replace Not-a-Number entries (NaNs),
  - Find and replace missing data, possibly supplement with interpolation,
  - Detect duplicates and possibly remove,
  - Check and correct lengths of time series.
- **Normalization.** Normalization rescales the data to a different numerical range. It defines the scale on which we want to work. Especially when comparing variables, this intervention is helpful. For machine learning methods, it can play a major role whether the data is passed in the correct scaling or not.
- **Filter.** In signal theory, filters play an important role. Machine learning methods can also benefit from this filter, as they help to smooth data (moving average) or to enhance certain properties of data (e.g., through the first and second derivative).
- **Triggering.** Especially with long time series with recurring signal sequences, triggering is often necessary. The repetitive part is then cut out, can be overlaid and examined for outliers.
- **Application of a function.** Sometimes it can be helpful to transform the data variables via a function. There are many variants here; the formation of the

logarithm, to better recognize the dynamics in exponents from exponential dependencies, is an example of this.

- **Transformation.** Transformations are comparable to normalization, but they also change the qualitative course of the variables or transfer them into a perspective that opens up a better, perhaps even low-dimensional, view. The goal of every transformation is to improve the later evaluability.
- **Statistical and process-related characteristics.** Here we determine important points in the course of data. These can be extremes, averages or other statistical sizes, but also positions motivated by process understanding, which are important for a technical reason. Examples are breakthrough points in air filters, saturation points, reaching half-lives or turning points in motion sequences.
- **Application of further algorithms.** In principle, the difference between evaluation methods and preprocessing is so fluid that we can consider almost any algorithm as data preprocessing, as long as it can be used sensibly. Especially the unsupervised learning methods, of which we will get to know the principal component analysis, K-means clustering method and the autoencoder, are excellently suited for data preparation.

## 3.2 Data Cleaning

### *3.2.1 Removal of Faulty Data Points*

A typical task of data preprocessing is data cleaning. Analysis algorithms cannot function properly with damaged or incomplete data. An example of this is the occurrence of Not-a-Number entries (NaNs). There are several reasons why the data contains NaNs: either because bad signals were already generated by the sensor or simply because the signals had a problem at some stage of processing.

In many raw data, you find such faulty values, missing values or wrong scalings. Where do these problem spots come from? The path that an entry in a database or in a file takes can be complex. Errors can occur during measurement, for example, when the measured value leaves the prescribed range. They can occur when saving the data, if the write operation was wrong. When working with databases, it may be that the query is not correct or an error occurred during the transfer of data over the network.

Always clean up at the source first
When confronted with data errors, you should find out where they come from. Which process was responsible for the errors? Often, data errors indicate another, deeper problem. Once you have identified this process, check whether the writing of NaNs or the causal error in general can be prevented.

**Example: Weather Station**
A weather station in your garden records the humidity and air pressure. It is powered by a solar cell. The data is transmitted to a computer via Bluetooth. The computer writes at the same rate, e.g. $1/s$, and it writes NaNs as soon as the transmitter does not send a value. Whenever the voltage of the solar cell is insufficient, the transmission is interrupted. Therefore, you receive a series of data with NaNs, in which a meaningful value for their measurements always occurs when the voltage was sufficient. In this case, it would be helpful to first improve the voltage of the measuring device and optimize the transmission chain as much as possible. ◀

**Example: QR Scanner**
Due to a dirt spot on a QR scanner (camera), the QR code of products is no longer read correctly. From a certain point in time, there are erroneous data in your database. Here too, it makes sense to remedy the situation by physically cleaning the camera. ◀

These examples are intended to show you how important the quality of the measurement chain is for the quality of the data. Do not try to correct errors in a measurement chain by repairing digital data. This would mean addressing the error in the wrong place. In industrial companies, agreeing on fixed maintenance and cleaning measures can also help to improve data quality.

Cleaning of raw data
Correcting problem areas directly at the data source is therefore, according to our previous considerations, always the best way to ensure data quality. However, in practice we often find ourselves confronted with historical data sets whose purely sensory information we can no longer improve afterwards. Or we technically have no real access to repair the measurement chain, e.g. when sensors are built in inaccessible. Ultimately, and this is usually the most important argument, the necessary correction of the sensor or the measurement chain may simply be uneconomical. In such cases, we must perform an adequate cleaning of the data on our side.

In Listing 3.1, a table is created and then re-indexed with Pandas so that it contains more rows than there are data. The only reason for this re-indexing is the creation of rows with NaNs.

**Listing 3.1** Replacing NaNs in data

```
import numpy as np
import pandas as pd

frame=pd.DataFrame(np.random.randn(4,3),
                   index=[1,2,4,7],columns=['A','B','C'])

# create NaNs "artificially" by expanding the number of
# rows of the data frame
frameWithNaNs = frame.reindex([1,2,3,4,5,6,7])
replacedNaNs = frameWithNaNs.replace({NaN:0.0})

print(frameWithNaNs)
print(replacedNaNs)
```

If you run the above code from Listing 3.1, you get the output 3.2. Rows 1 to 8 contain the original matrix with NaN values. Rows 9 to 16 show the same matrix, but with the replacement NaN = 0.0.

**Listing 3.2** Output of Listing 3.1

```
          A         B         C
1  1.015877 -0.194974 -0.777067
2  0.199423 -1.477063  0.679932
3       NaN       NaN       NaN
4  2.039111  0.908888  0.695052
5       NaN       NaN       NaN
6       NaN       NaN       NaN
7 -1.253851  0.255705 -0.569040
          A         B         C
1  1.015877 -0.194974 -0.777067
2  0.199423 -1.477063  0.679932
3  0.000000  0.000000  0.000000
4  2.039111  0.908888  0.695052
5  0.000000  0.000000  0.000000
6  0.000000  0.000000  0.000000
7 -1.253851  0.255705 -0.569040
```

Such replacements are of course not only possible with the toolbox Pandas. It is used here only as an illustrative example. You should also handle the replacement of data very consciously. The above code increases the number of occurrences of 0.0. If this number is relevant for your evaluation, e.g. if you are conducting a statistical evaluation, then such a replacement can also lead to errors.

The choice of the right replacement depends on the specific situation. Often you want to insert a numerical marker in the data to trace where the NaNs occurred. Suitable for this are numerical values that do not occur in the regular value range of the respective sensor. For a temperature sensor that measures between −50 °C and 100 °C, −1000 °C would be a suitable marker.

### 3.2.2 Missing Data

Should data be incomplete, e.g., because there is an incorrect value at an index position or a NaN replacement has been made, the data set can be completed by direct interpolation. Let's assume that the defective data position is at $x_i$. If both $x_{i-1}$ and $x_{i+1}$ contain trustworthy values, we can use

$$x_i = \frac{x_{i-1} + x_{i+1}}{2} \tag{3.1}$$

to implement a manual interpolation. The mean value represents the most probable value at exactly this position.

### 3.2.3 Tracking and Eliminating Duplicates

With the following simple listing, we can effectively search for duplicates in our data frame. To practice this, we have created a very simple set of vectors (without any meaning) to see how Pandas enables the check for duplicates. The listing 3.3 shows an example of searching for duplicates.

**Listing 3.3** Searching for duplicates using Pandas

```
import numpy as np
import pandas as pd

x0 = [1,2,3,4,5,6,7,8,9]
x1 = [1,3,1,4,4,1,2,5,1]
x2 = [1,3,1,4,4,3,2,5,5]
x3 = [1,3,3,3,1,2,4,5,1]

frame = pd.DataFrame([x0,x1,x2,x1,x3])
frame.duplicated()
```

As a result of the search in the Pandas DataFrame, we get in listing 3.4 a vector with True/False entries, which shows us whether and where the duplicate can be found.

Listing 3.4 Result of Listing 3.3

```
0    False
1    False
2    False
3     True
4    False
dtype: bool
```

Due to the constructed example, it turns out that the vector `x1` occurs exactly twice and the duplicate is at the penultimate position. In fact, we have inserted $\boldsymbol{x}_1$ twice into the data frame, at position $i = 1$ and at position $i = 3$.

## 3.3 Normalization

### *3.3.1 Reasons for Normalizing Data*

In Chap. 1 we discussed scales and scale levels discussed. They help us understand the nature of a data series. The data type also determines the possible mathematical operations that we can perform with it. In the example of the different relative temperature differences in the Celsius and Kelvin scales from 1.2.7 we could already see how important the measurement range is for our own perception. This effect depends on the scaling of our data and we can influence this with the help of suitable rescaling.

**Example: Air Pressure**
Let's take another example to help clarify this. Depending on the altitude, the air pressure varies between 1070 hPa and 800 hPa (on high mountains). So, at 800 hPa we would speak of very low pressures and at 1070 hPa of very high pressures. If our data refers to tire pressures, which are rather in the range of 200 kPa, we have a unit jump from hPa to kPa. Our previous assessment of low and high would be obsolete. ◀

### *3.3.2 Types of Normalizations*

Often, the scaling of our data needs to be adjusted between different datasets. Data is normalized to become comparable. The following normalizations help with this:

- **Normalization to the maximum of the data vector.** The data is divided by its maximum,

$$\boldsymbol{x} = \frac{\boldsymbol{x}}{\max(|x_i|)}, \tag{3.2}$$

  which results in a scaling of the individual data points within the interval $[-1, 1]$.

- **Normalization to the sum of the data vector.** The data is divided by its total sum (magnitude of the data vector),

$$\boldsymbol{x} = \frac{\boldsymbol{x}}{\sum_i x_i}, \tag{3.3}$$

  which results in each data vector having the same area after normalization.
- **Min-Max Normalization.** Sometimes it is necessary to only consider positive data points for an evaluation. Then our data can be mapped to the values [0,1] by first subtracting the minimum value and then scaling to the largest distance,

$$\boldsymbol{x} = \frac{\boldsymbol{x} - \mathbf{1}\min(\boldsymbol{x})}{\max(\boldsymbol{x}) - \min(\boldsymbol{x})}. \tag{3.4}$$

- **Subtraction of the mean.** Another form of normalization ensures that our resulting data has a new average of $\langle \boldsymbol{x} \rangle = 0$. Here, the mean is first subtracted and then divided by the distance between maximum and minimum,

$$\boldsymbol{x} = \frac{\boldsymbol{x} - \mathbf{1}\langle \boldsymbol{x} \rangle}{\max(\boldsymbol{x}) - \min(\boldsymbol{x})}. \tag{3.5}$$

Normalization is a common source of error and should therefore be carried out with care. One can unintentionally change the meaning of data and in the worst case reduce or lose the information content of variables. However, normalization is necessary for the later field of physically informed machine learning. It helps to focus on relevant aspects of the data. Important areas of measurements are specifically highlighted.

### *3.3.3 Application of Normalization*

To emphasize the importance of normalization, we would like to consider a simple example that will accompany us through the rest of the book.

**Example: Motor Current Anomaly**
You are looking at a composite of motors. The motors are identical and provide you with current characteristics. These characteristics contain information about whether the process ran correctly or not. The latter is comparable to an impending defect of a motor or problems with their application. However, an error occurred during the recording of the data and the automatic detection of the measuring range measured the current in milliamperes in some cases and in A in other cases. ◀

We first load the data and look at it as we have outlined in Listing 3.5.

Listing 3.5 Loading an example data file and plotting the contained data

```
import matplotlib.pyplot as plt
import numpy as np
import pickle

data = pickle.load(open('EX03EngineExt.pickle', 'rb'))
for i in range(0,500):
    plt.plot(data['X'][i], color='k', alpha=0.25)

plt.xlabel('t', fontsize=18)
plt.ylabel('X', fontsize=18)
plt.tick_params(labelsize=18)
```

The result is shown in Fig. 3.2: some current profiles are clearly visible, while another group appears as a horizontal line in the diagram. The latter are exactly the data that were measured in the ampere range, so they have much smaller values than the measurements in mA. We have not yet noted a unit on the axis, as we are only working purely exploratively here and are initially only looking at the data.

We want to normalize the data so that all characteristics have the same order of magnitude. Therefore, we change our existing code by inserting Listing 3.6 from line 5. This shows an extremely simple, but effective, manual triggering—a preprocessing step that we will revisit in Sect. 3.5. Whenever the maximum of a curve is large enough, we let it apply, in all other cases we scale the curve by a factor of 1000 to switch from the A to the mA scale.

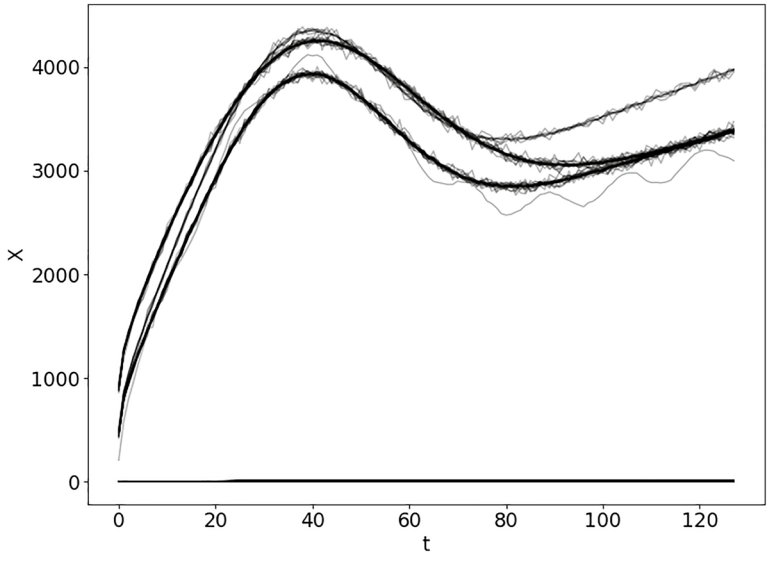


**Fig. 3.2** Result of Listing 3.5

**Listing 3.6** Applying a simple trigger

```
data = pickle.load(open('EX03EngineExt.pickle', 'rb'))
newData = []
for i in range(0,500):
    if np.max(data['X'][i]) > 1000:
        newData.append(data['X'][i])
    else:
        newData.append(1000*data['X'][i])

    plt.plot(newData[-1], alpha=0.2)
```

The example is intended to show you how easy it can be to fix the error of the measuring ranges. However, it is not always clear that such an error exists. We want to briefly present the other normalization variants.

Listing 3.7 shows the procedure to normalize to the sum of the data vector. Here we go through the same for-loop as for the correction of the units.

**Listing 3.7** Sum-Normalization

```
sumNormalizedData = []
for i in range(0,500):
    normalised = newData[i] / np.sum(newData[i])
    sumNormalizedData.append(normalised)
    plt.plot(sumNormalizedData[-1], alpha=0.2)
```

If it is important to normalize to the maximum, Listing 3.8 shows how the reference to the maximum per data vector is calculated with minimal changes.

Listing 3.8 Maximum Normalization

```
maxNormalizedData = []
for i in range(0,500):
    normalised = newData[i] / np.max(newData[i])
    maxNormalizedData.append(normalised)
    plt.plot(maxNormalizedData[-1], alpha=0.2)
```

Finally, there is a normalization where we first subtract the mean and then normalize to the maximum. This is shown in Listing 3.9. But let's not confuse this variant with a normalization to the resulting maximum after subtraction.

**Listing 3.9** Subtraction of mean

```
meanMaxNormalizedData = []
for i in range(0,500):
    normalised = (newData[i] - np.mean(newData[i])) / np.max(
        newData[i])
    meanMaxNormalizedData.append(normalised)
    plt.plot(meanMaxNormalizedData[-1], alpha=0.2)
```

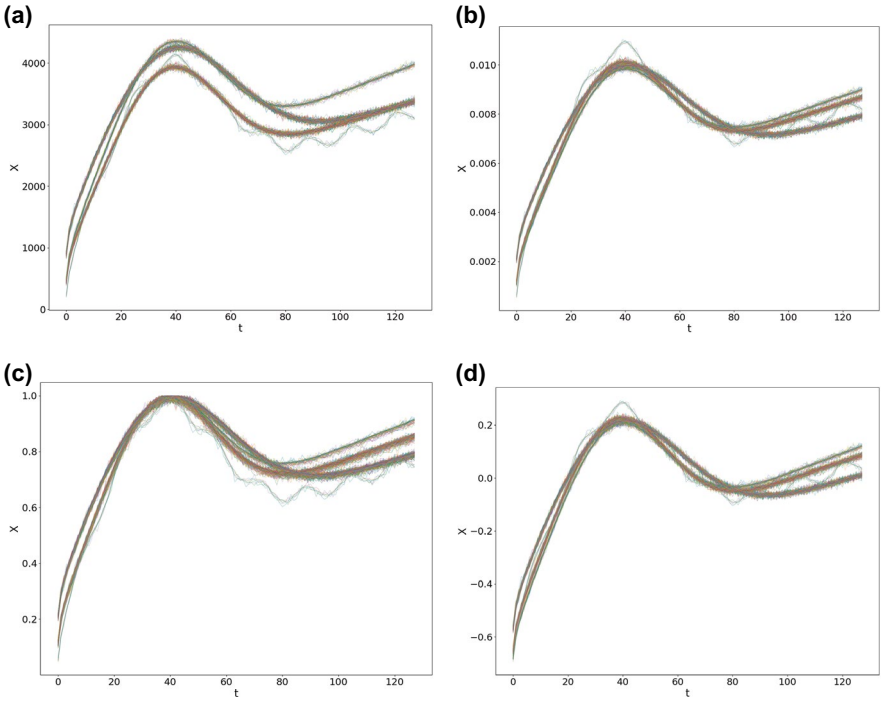


**Fig. 3.3** Impact of different normalizations on the motor current dataset. (a) No additional normalization, (b) Normalization to the sum (3.3), (c) Normalization to the maximum (3.2) and (d) Subtraction of the mean and normalization to the maximum

In Fig. 3.3 the normalization variants shown here are presented. Each normalization has a different effect. With some normalizations, you can amplify or attenuate effects. Whether this is useful or whether the normalization is disadvantageous for your data analysis depends on the specific example.

## 3.4 Filtering of Data

### *3.4.1 Moving Average*

One of the most important transformation steps in data processing and data preparation is smoothing. Data that is too noisy must be smoothed so that subsequent analysis algorithms do not confuse the inherent noise with the actual information content. The classic filter for denoising a signal is the moving average. In Listing 3.10, an example implementation of this filter is shown. We examine the effect of this filter using a simple example:

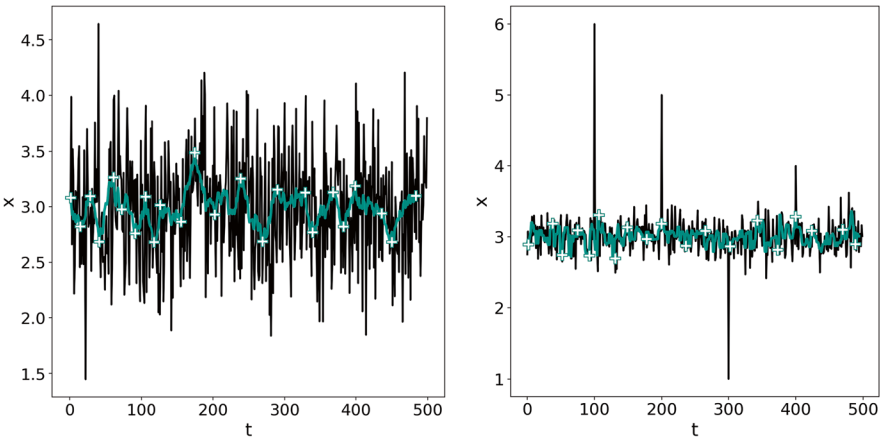


**Fig. 3.4** Effect of the moving average (left) and effect of the moving median filter (right). The black line shows the raw data, the green (gray) line with white crosses shows the smoothed data

**Listing 3.10** Moving average filtering

```
1 import numpy as np
2
3 x = 3+0.5*np.random.randn(500)
4 filtered_x = []
5 window = 15
6
7 for i in range(0,len(x)-window):
8     filtered_x.append(np.mean(x[i:i+window]))
```

The variable `x` is a combination of 3.0 and a random number between −0.5 and 0.5. `x` is to be transformed into the filtered variable `filtered_x`. For this, we use the for-loop in lines 7 and 8. Here, the mean of `np.mean(x[i:i+15])` is first calculated and then this value is stored in `filtered_x`.

This specific implementation calculates the mean always forward. The for-loop must take this into account and therefore stops in this case 15 values before the end. With the code in Listing 3.11, the effect of the filter can be observed. Fig. 3.4 shows on the left side the noisy signal in black and the moving filter value in green. The noise is effectively smoothed.

**Listing 3.11** Plotting the moving average

```
import matplotlib.pyplot as plt
plt.plot(x, 'k', linewidth=2.0)
plt.plot(filtered_x,'-',color=[0.1,0.65,0.6],linewidth=3.0,
    alpha=0.9)
plt.xticks(fontsize=18)
plt.yticks(fontsize=18)
plt.xlabel('t', fontsize=20)
plt.ylabel('x', fontsize=20)
```

The filter is described by a parameter, which is the window size, in Listing 3.10 named by the variable `window`. The larger the window, the stronger the smoothing effect, as the average is taken over a larger number of points.

### *3.4.2 Convolution*

The sliding average filter shown here can be implemented more simply by using the convolution operation. This is exemplified in program example 3.12, with a smoothing window of sample length 15. For this, we use the function `np.ones(N)`, which generates an array with N ones for us. This shaping function, in our case a series of ones, is also called a convolution kernel.

**Listing 3.12** Moving Median Filter

```
x = 3+0.5*np.random.randn(500)
window=15
filtered_x = np.convolve(x,
        np.array(np.ones(window)/window,
        mode='valid')
```

Let's take a closer look at this property for the moment. The convolution also helps us to perform other processing stages of our data quickly.

**Convolution** Formally, the convolution operation $\circ$ for two continuous functions $x(t)$ and $y(t)$ is defined by

$$(x \circ y)(\tau) = \int x(t)y(\tau - t)dt. \tag{3.6}$$

In the case of two data vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ the discrete convolution operation is defined as

$$(\boldsymbol{x} \circ \boldsymbol{y})_i = \sum_k x_k y_{i-k}. \tag{3.7}$$

The moving average filter, as discussed in the previous section, can be expressed extremely compactly using

$$\mathrm{MA}(x) = \boldsymbol{f} \circ [1, 1, 1, \ldots, 1]/N, \tag{3.8}$$

which was also implemented in code example 3.12.

### 3.4.3 The Median Filter

The median $m(\boldsymbol{x})$ of a vector $\boldsymbol{x}$ with sample data points is defined as the value $x$, which separates the sample so that 50 % of all points lie below $x$ and the other 50 % of all points lie above $x$. Formally, the median can be written as

$$m(\boldsymbol{x}) = \begin{cases} x_{(n+1)/2}, & \text{if } n \text{ is odd,} \\ \frac{x_{n/2} + x_{(n/2)+1}}{2} & \text{if } n \text{ is even,} \end{cases} \tag{3.9}$$

where $n$ is the length of the vector $\boldsymbol{x}$. One of its most important properties is to remove extreme outliers from data sets. Consider the following example,

$$\mu([1, 1, 5, 1, 1]) = 2.6,$$
$$m([1, 1, 5, 1, 1]) = 1.0.$$

Given the peculiar vector $\boldsymbol{x} = [1, 1, 5, 1, 1]$, the median $m$ apparently completely ignores the influence of the highest number 5.

We can also use the median shown in 3.4.3 to filter the data series by applying it in the same way as the mean within a window on the data.

**Listing 3.13** Gleitender Median-Filter

```
%pylab
import matplotlib
import numpy as np

x = 3+0.2*np.random.randn(500)
x[100]=6
x[200]=5
x[300]=1
x[400]=4
filtered_x = []

for i in range(0,len(x)-5):
    filtered_x.append(np.median(x[i:i+5]))
```

With the code in 3.11, the result of Listing 3.13 can also be displayed. In Fig. 3.4, the effect of the moving median filter is shown on the right side.

## 3.5 Triggering

### *3.5.1 Time series and repetitive data*

There are technical processes that record their data in long time series. Often, however, the actually interesting time range is only a localized phenomenon. Large areas of the data series are therefore not relevant at all. In such situations, one would like to cut out the relevant sequences and examine them specifically. This process is called triggering. A trigger is a switch that engages under a certain condition and records the data along with it.

In Fig. 3.5, an example of such data is shown. The data for the example was synthetically produced, but is strongly based on real situations such as the rolling process, the switching on and off of circuits, or laser activations.

### *3.5.2 Implementation of the trigger*

In Listing 3.14, we have implemented a trigger as an example. The trigger is tested on the dataset `EXAMPLE02.pickle`. Often, data recording tools already offer trigger functionality. The method shown here serves for understanding. The function is passed a time series $x$. In addition, the parameters `threshold`, i.e., a limit value for $x$, and `windowLength`, the length of the triggered area, are to be passed. Whenever $x$ exceeds the limit value, a corresponding window is cut out. This general form of the trigger can be rewritten for any triggering factors or conditions. It is therefore extremely easy to adapt to other problems.

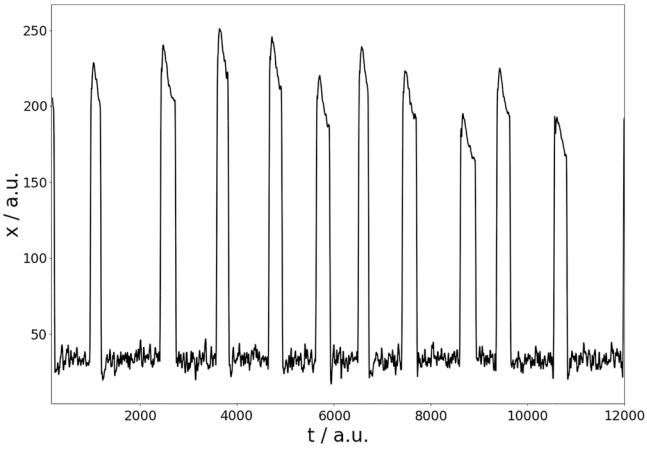


**Fig. 3.5** Time series with repetitive process behavior

**Listing 3.14** Triggering on a continuous signal

```
x = pickle.load(open('EXAMPLE02.pickle','rb'))

def trigger(x, threshold, windowLength):
    triggerBuffer = []
    newTriggeredSection = []
    triggerStart = False
    isTriggerOn = False
    q=0
    for i in range(0,len(x)):

        if x[i]>=1*threshold:
            isTriggerOn = True
            q=0

        if isTriggerOn:

            # -->
             #if len(newTriggeredSection) < windowLength:
              # newTriggeredSection.append(x[i-50])

            # -->
             newTriggeredSection.append(x[i-50])

             q+=1

        if q>= windowLength+50:
             q = 0
             isTriggerOn = False
             newTriggeredSection = []
             triggerBuffer.append(newTriggeredSection)

    return triggerBuffer

data = trigger(x, threshold=70, windowLength=350)
```

You can also easily improve the implementation. It is often not necessary to iterate through the entire data series $x$. Whenever the trigger switches to True, you could directly jump to the end of the window length and continue iterating from there. Depending on what the specific problem looks like, there are therefore much more efficient trigger approaches.

The result of the triggering is shown in Fig. 3.6. Here, the data was also normalized to the maximum in order to better overlay the curves. You can already see from this type of representation whether individual curves show different behavior. A slight change in the code in 3.14, marked by the arrows in the comment, results in all windows being the same length.

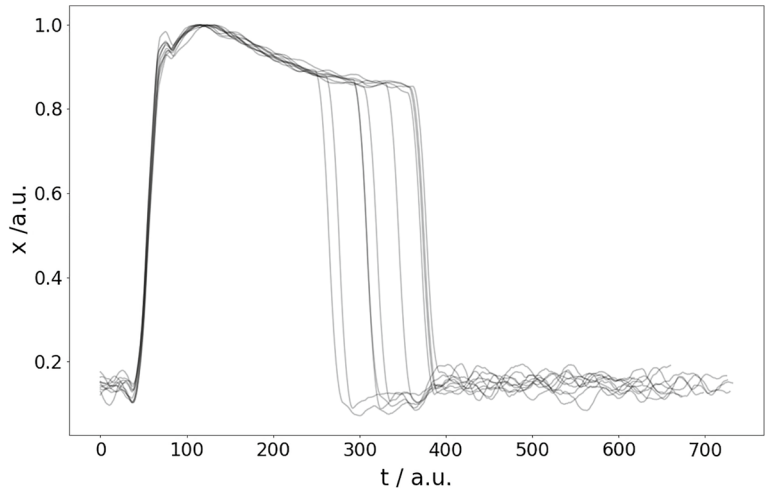


**Fig. 3.6** Triggered data, each repetitive process step is represented by a curve

## 3.6 Transformations

### *3.6.1 Differentiation of Data*

The derivative $f'(x)$ of a function $f(x)$ gives as is well known the slope at the point $x$. If a function is constant in $x$, the derivative is 0. In data, the derivative thus helps to eliminate constant parts and to amplify increases, decreases, any form of variation. The second derivative is a further stage of this procedure.

With the help of the convolution introduced in (3.7), the first derivative can be calculated over

$$\frac{df}{dx} = \boldsymbol{f} \circ [-1, 1] \tag{3.10}$$

If you apply the convolution kernel $[-1, 1]$ one more time, the second derivative results,

$$\frac{df}{dx} = \boldsymbol{f} \circ [1, -2, 1]. \tag{3.11}$$

If you are faced with particularly noisy data, it is advisable to combine the derivative with a smoothing average. Since deriving determines the slopes and noisy data practically consist of many slopes, you would otherwise amplify the noise. The combination of smoothing and derivation is

$$\text{Filter}(x) = \boldsymbol{f} \circ [-1, -1, -1, -1, 1, 1, 1, 1]/4, \tag{3.12}$$

where we have freely chosen the smoothing length 4. It must be adapted to the respective problem. Souza et al. show in [2] and how the application of such preprocessing can be systematically integrated into the process of data mining. Especially the differentiation is used in [5] to highlight the visibility of effects for machine learning methods.

The effect that a simple differentiation has on a data vector is shown in Fig. 3.7. The associated listing 3.15 shows how this example was constructed. We use the function `diff(y,n)` for the differentiation, where $y$ represents the input data and $n$ the degree of differentiation.

**Listing 3.15** First and second derivative using Numpy

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0,100)
y1 = 0.5* np.exp(-(x-40)**2/25)
y2 = 0.2* np.exp(-(x-30)**2/25)
y3 = y1+y2+0.1*x

fig, (ax1, ax2, ax3) = plt.subplots(1,3,figsize=(16,9))
ax1.plot(x,y3,'k', linewidth=2.5)
ax2.plot(np.diff(y3),'k', linewidth=2.5)
ax3.plot(np.diff(y3,2),'k', linewidth=2.5)
```

In the left diagram of Fig. 3.7 you can hardly recognize the two maxima of the Gaussian function. The linear course of the curve dominates. The first derivative reduces this course. In the given example, the slope was 0.1 and the diagram in the middle of the figure shows the derivative, whose constant parts start at 0.1. A second derivative transforms this into a course around 0, because the constant part is eliminated by the further derivative. If these data were associated with a technical process, you would best highlight the effect shown here with derivatives.

### 3.6.2 *Functional Transformation*

Of course, you can apply any function to a data vector. This is always useful when you already suspect a certain trend,

$$\boldsymbol{x} \mapsto \tilde{\boldsymbol{x}} = \mathcal{F}(\boldsymbol{x}; \boldsymbol{\pi}), \tag{3.13}$$

where $\boldsymbol{x}$ stands for your original data series, $\mathcal{F}$ can be any transformation function and $\boldsymbol{\pi}$ captures the parameters of this function. The exponential function is a good example here. If you find exponential trends in your data, the logarithm is obviously a suitable transformation. This way, as when plotting on logarithmically scaled paper, you transform your data into a quasi-linear representation. In this case, no parameters would be necessary.

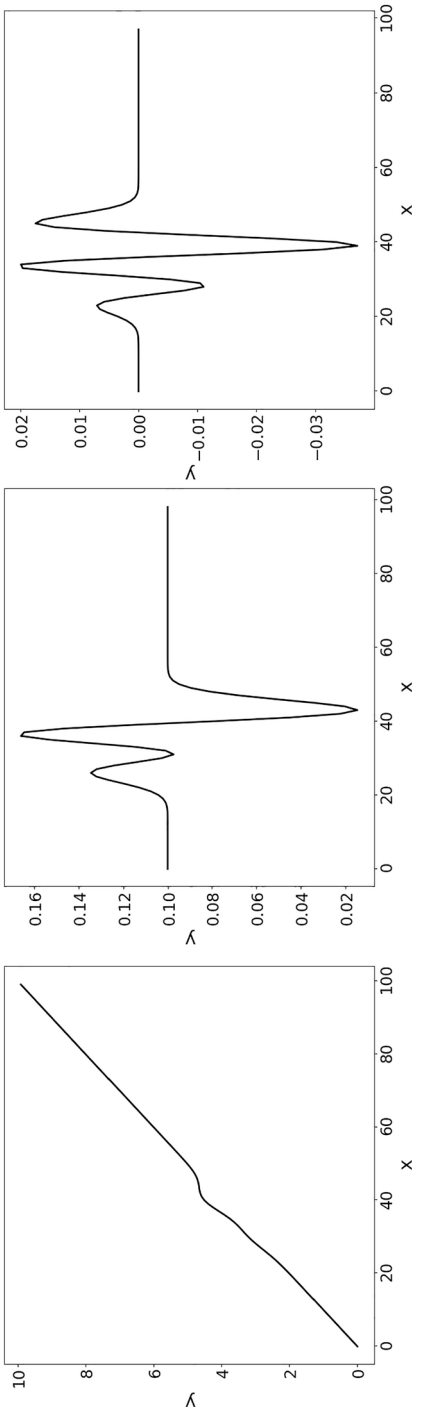


**Fig. 3.7** Effect of the derivative on a data vector. Left: original course. Middle: first derivative. Right: second derivative

Applying a functional transformation is important for the later discussed physically-informed learning methods and for the discussion of sensitivity analyses. Physically-informed learning uses such transformations to move from one input variable to an alternative variable. Sensitivity analysis can use this form of transformation in trained networks to check the analytical dependency between the result of a learning method and its input variables.

The following notes are important from a practical point of view for applying a direct function to your data:

- Do not increase complexity. For example, do not artificially create an oscillation or another complex trend from the constant trend of your variables. This would contradict the purpose of preprocessing and above all only unnecessarily complicate your further processing steps.
- Avoid many parameters. Each parameter you introduce here acts as an additional adjustment screw in the learning methods. And since learning methods already bring many parameters with them, this greatly increases the depth of variation.
- If you try such a transformation, beware of new defective data after applying the function. If your dataset contains many zeros, divisions can lead to NaNs because you cannot divide by zero. Roots of negative values lead to complex results that cannot be used by every downstream method.

### *3.6.3 Fourier Transformation*

Often we find recurring patterns in data, e.g. oscillations. They are characteristic of the signal trend. To make these patterns more visible, we can highlight special aspects from the signal through mathematical preprocessing.

The Fourier transformation calculates the frequencies and amplitudes of these signal components—it transforms from the space of temporal trends to the frequency space (also called Fourier space). For engineering sciences or for natural sciences in general, it is a known tool to describe oscillatory phenomena and to quickly switch between time and frequency spaces (Fig. 3.8).

The **Fourier Transform** $\mathcal{F}[x(t)](\nu)$ of a function $x(t)$ in the continuous case is

$$\mathcal{F}[x(t)](\nu) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} x(t) \exp(-i2\pi\nu t) dt \tag{3.14}$$

and in the discrete case, with discrete time steps $\Delta t$,

$$\mathcal{F}(\nu) = \frac{1}{\sqrt{2\pi}} \sum_{k=0}^{N-1} x(k\Delta t) \exp(-i2\pi\nu k\Delta t). \tag{3.15}$$

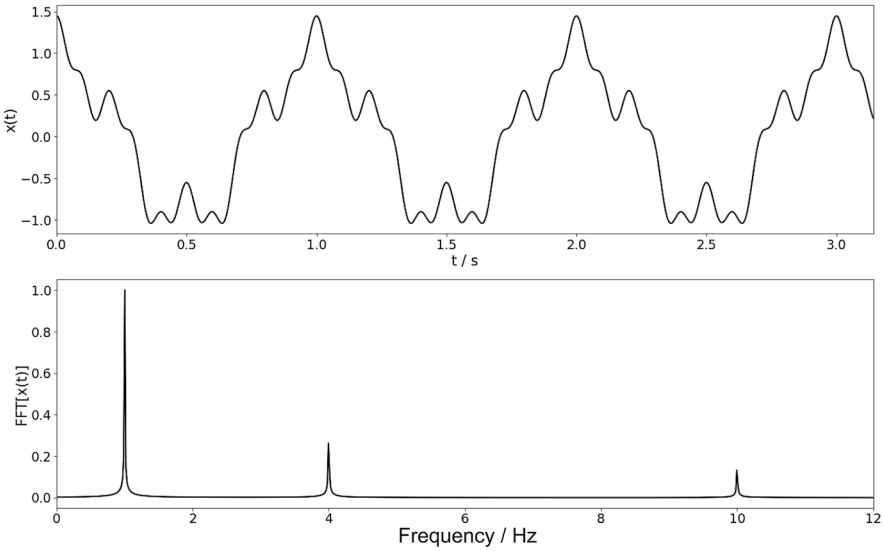


**Fig. 3.8** Fourier transform from our code example with self-generated data

It develops the original course of the data into a series of oscillations $\exp(-i2\pi\nu k\Delta t)$. Here, $i$ stands for imaginary numbers and is defined via $i^2 = -1$. Each oscillation is uniquely characterized by its frequency $\nu$: They are localized in Fourier space, i.e., assigned to exactly one point on the frequency axis.

Since the calculation of the Fourier transform is time-consuming on a computer—the algorithm for this would be of the order $(n^2)$ –, a method was developed which keeps the number of calculation steps for determining 3.15 small. This method is called Fast Fourier Transform (FFT). It divides the original time series into two new, smaller series using the individual time steps: the odd time steps go into one, the even time steps into the second new series. Both newly created time series are shorter than the original and can be transformed individually. Of course, the division process repeats recursively. The procedure thus created is called the Cooley-Tukey algorithm and is currently the most commonly used variant of the FFT.

For our considerations, the details of the FFT are actually not crucial. They are only intended to serve as information. The actual execution of such an algorithm is already included in the package functions in Python and can be applied by us very easily and quickly.

In Listing 3.16 you will find an application example of the Fourier transformation on a simple combined oscillation. We have also added the code that represents the transformation. With this example, you can quickly examine transforms of various input data yourself. In many cases, the Fourier transform can provide a new

perspective that improves the view of the data. This is the case even when there are no obvious, clean oscillations in the data. Often, it is then other recurring patterns that can be well recognized in the frequency space.

**Listing 3.15** Fast Fourier Transform (FFT)

```
import matplotlib.pyplot as plt
import numpy as np

# Definition der Frequenzen in Hz
nu1 = 1
nu2 = 4
nu3 = 10

A1 = 1
A2 = 0.3
A3 = 0.15

# Beispieldaten mit diesen Frequenzen
final_t = 20*np.pi
dt = final_t/2**14
t = np.arange(0,final_t,dt)  # in s
x = A1*np.exp(1j*2*np.pi*nu1*t) \
    + A2*np.exp(1j*2*np.pi*nu2*t)\
    + A3*np.exp(1j*2*np.pi*nu3*t)\

# Berechnung der FFT
fft = np.fft.fft(x)
freq = np.fft.fftfreq(x.size, d=dt)

# Normalisierung
fft = np.abs(fft)
fft /= np.max(fft)

# Darstellung
fig, (ax1,ax2) = plt.subplots(2,1,figsize=(16,9))

ax1.plot(t, np.real(x),'k', linewidth=2.0)
ax1.set_xlabel('t / s', fontsize=20)
ax1.set_ylabel('x(t)', fontsize=20)
ax1.set_xlim([0,np.pi])
ax1.tick_params('both',labelsize=18)
ax1.tick_params('both',labelsize=18)

ax2.plot(freq[0:8191], fft[0:8191],'k', linewidth=2)

ax2.set_xlim([0,12])
ax2.set_xlabel('Frequenz freq / Hz', fontsize=20)
ax2.set_ylabel('FFT[x(t)]', fontsize=20)
ax2.tick_params('both',labelsize=18)
ax2.tick_params('both',labelsize=18)
```

Listing 3.17 finally shows the application of the Fourier transform to our engine example. For this, we use the intermediate state reached in Listing 3.17 (the variable `newData`) and calculate an FFT for each data vector.

**Listing 3.15** Applying the FFT to the engine example

```
for i in range(0,500):
    fft = np.fft.fft( (newData[i]-np.mean(newData[i]))/np.sum(
        newData[i]))
    fft = np.abs(fft)
    fft /= np.sum(fft)
    if fft[8]>0.015:
        myColor = 'r'
    else:
        myColor = 'k'
    plt.plot(fft, color=myColor, alpha=0.2)
```

Since the FFT actually calculates complex numbers, all results of `np.fft.fft(x)` are complex: they contain a real part and an imaginary part. For this reason, we deliberately chose the complex notation above. You could extract the individual components of a complex number $x$ via `np.real(x)` or `np.imag(x)` or determine their (real) magnitude via `np.abs(x)`. We used the latter in the example.

The results of this short code are illustrated in Fig. 3.9 and show that a specific frequency stands out. We have deliberately marked this frequency, let's call it $\nu^*$, in red during preprocessing. All transformations where we see more pronounced values for $\nu^*$ correspond to the original data with the easily recognizable oscillation. We thus have an easily verifiable criterion whether an oscillation is present or not.

This example of exploratory data viewing, testing a transformation, and ultimately finding suitable criteria to find a disturbance are often recurring steps when working with process and machine data.

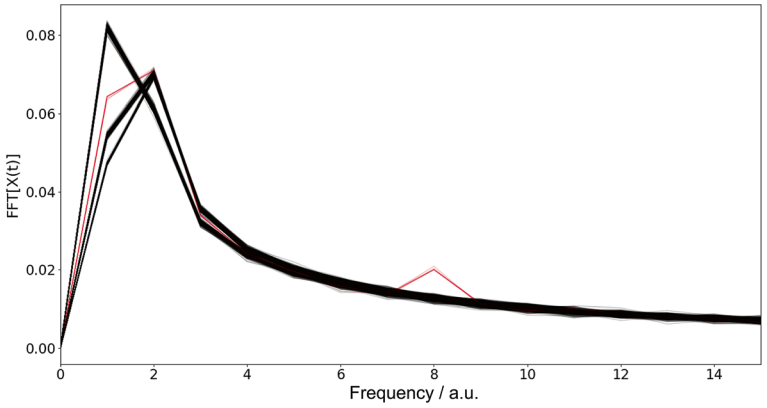


**Fig. 3.9** Fourier transform of our motor current example. One curve shows an increase at the dimensionless frequency 8

### 3.6.4 Wavelet Transformation

In some situations, an oscillatory behavior only occurs in a time window. So it is temporally local. However, the FFT represents our data through oscillating functions, and these are temporally continuous. A better alternative is to choose a projection that can reflect this temporal limitation. This is exactly where the so-called wavelets come into play.

Wavelets are temporally limited wave patterns with a width $a$ and a temporal position $b$. A simple and easily understandable example is the Ricker wavelet. It is the second derivative of the Gaussian function,

$$\psi\left(\frac{t-b}{a}\right) = \frac{d^2}{dt^2}\frac{1}{\sqrt{\pi a^2}}\exp\left(-\frac{(t-b)^2}{a^2}\right). \tag{3.16}$$

The continuous wavelet transformation CWT$[x(t)]$ uses this wavelet basic form $\psi$ to obtain a width- and position-dependent projection of the function $x(t)$.

The **continuous wavelet transformation** of a function $x(t)$ with respect to a wavelet $\psi(\frac{t-b}{a})$ is given by

$$\text{CWT}[x(t)](a,b) = \frac{1}{|a|^{\frac{1}{2}}}\int_{-\infty}^{\infty} x(t)\psi\left(\frac{t-b}{a}\right)dt. \tag{3.17}$$

Note the form of the Continuous Wavelet Transformation result, which depends on $b$ and $a$. Unlike the FFT, which contains the amplitude as a function of frequency in the result, the wavelet transformation provides an amplitude as a function of $b$ and $a$, thus a 2-dimensional result. In this form, the CWT thus increases the dimension of the input data. Listing 3.18 first demonstrates the creation of artificial sample data to understand the effect of the CWT.

**Listing 3.18** Synthetic example data for the wavelet analysis

```
import numpy as np
import scipy.signal as sp

final_t = 5*np.pi
dt = final_t/2**14
t = np.array(np.arange(0,final_t,dt))
x = np.cos(2*np.pi*1.*t) + 3*np.exp(- 1*(t-1))
```

An exemplary application can be found in Listing 3.19, where we also present the 2D representation of the wavelet transformation.

**Listing 3.19** Continuous wavelet transformation

```
%matplotlib widget
widths = 1*np.arange(10,200,2)
cwtmatr = sp.cwt(x, sp.ricker, widths)

plt.imshow(np.real(cwtmatr), cmap='nipy_spectral', vmin=0,
           vmax=np.max(np.real(cwtmatr)),
           aspect='auto')
```

We now want to look at the example with the motor currents from the wavelet perspective. For this, we take the previous code example and supplement it in Listing 3.20, so that we can distinguish good and bad cases based on the Fourier transform. This serves for illustration in this case and is a good example of how the methods can be used in combination.

**Listing 3.20** Splitting engine data for the analysis with a CWT

```
Bad = []
Good = []
for i in range(0,500):
    fft = np.fft.fft( (newData[i]-np.mean(newData[i]))/np.sum(
        newData[i]))
    fft = np.abs(fft)
    fft /= np.sum(fft)
    if fft[8]>0.015:
        myColor = 'r'
        Bad.append(newData[i])
    else:
        myColor = 'k'
        Good.append(newData[i])
    plt.plot(fft, color=myColor, alpha=0.2)
```

We now have clearly defined sets for good and bad cases. This is a state that we must always bring about for classification tasks. If we have access to such a divided data set, the subsequent application of a classification algorithm is often simple and without problems. The prerequisite, of course, is that a criterion for distinction can really be found.

We use both sets, Good and Bad, in Listing 3.21, to determine the transforms for individual cases of these sets. The results of this code are summarized in Fig. 3.10. A criterion for the detection of the bad case can also be found in the CWT result.

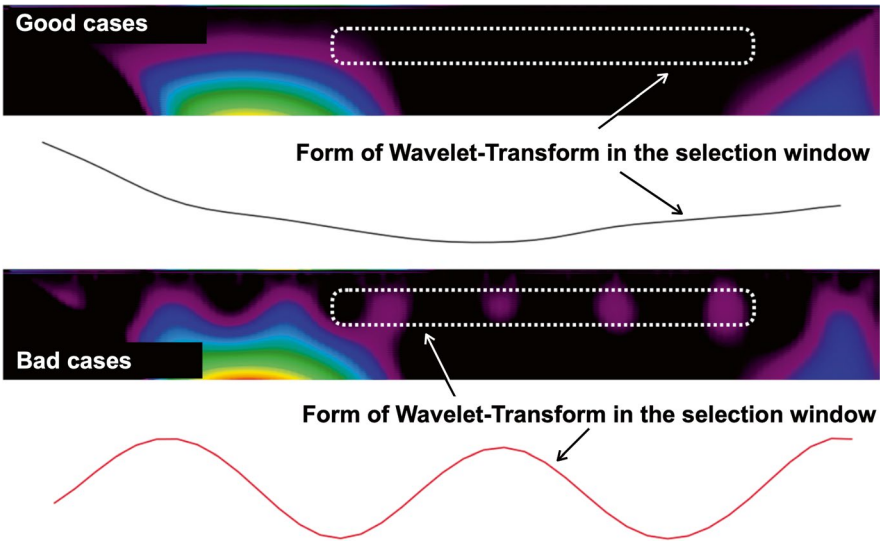


**Fig. 3.10** Wavelet transformation of a case from the good set (top) and a case from the bad set (bottom). By cutting out a suitable area in the transforms, a criterion for the set can be derived

**Listing 3.21** Continuous Wavelet Analysis using the example of engine data

```
import scipy.signal as sp
widths = 0.1*np.arange(1,100,.1)

goodCwtmatr = sp.cwt(Good[2], sp.ricker, widths)
badCwtmatr = sp.cwt(Bad[2], sp.ricker, widths)

fig, (ax1,ax2,ax3,ax4) = plt.subplots(4,1)
ax1.imshow(np.real(goodCwtmatr), cmap='nipy_spectral', vmin
    =0.0,
           vmax=0.01,
           aspect='auto')
ax2.plot(np.real(goodCwtmatr[300,50:90]),'k')
ax3.imshow(np.real(badCwtmatr), cmap='nipy_spectral', vmin
    =0.0,
           vmax=0.01,
           aspect='auto')
ax4.plot(np.real(badCwtmatr[300,50:90]),'r')
ax1.axis(False)
ax2.axis(False)
ax3.axis(False)
ax4.axis(False)
```

However, detection is more difficult than with Fourier analysis. It even seems to be a disadvantage to use the CWT, as the dimensionality of the evaluation increases. In fact, the benefit of transformation for the recognizability of a separating property is at the heart of almost every exploratory work.

## 3.7 Quantification of Stochastic Properties

### *3.7.1 Histograms*

We know some selected distributions from the previous chapter, but how can we extract these distributions from our data? Our goal is not only to have a good understanding of the process. Rather, we want to determine further characteristics from the data, with which we can reduce the dimension of the data or which can help a machine learning method to derive models from the data.

In this context, knowledge of the distribution is an important tool. Through the analytically calculable parameters of the distributions, we can define limit values, control the correctness of processes, and distinguish the quality of products. Therefore, we are interested in being able to extract the probability distribution from the data, and this is most easily done through histograms.

We determine through a histogram how often a specific value of a variable occurs. Sensibly, we choose for the frequencies contiguous ranges $I_k$, intervals that cover a certain range of values of the variable. To create a histogram, we count how often our variable passes through the different ranges $I_k$.

We say $x_i$ lies in the interval $I_k = ]x_{k,\min}, x_{k,\max}]$, $x_i \in I_k$, when $x_i < x_{k,\max} \wedge x_i > x_{k,\min}$ is fulfilled. We interpret $I_k$ as a set. We refer to the discrete representation of $x_k$ against the power of all sets $|I_k(x_k)|$ as a **histogram**.

In Listing 3.22 we show how to create a histogram yourself. This is intended for illustration. Our tools in Python offer simpler and faster ways to get this information.

**Listing 3.22** Continuous Wavelet Analysis using the example of engine data

```
import matplotlib.pyplot as plt
import numpy as np

x = 2*np.random.randn(1000)
k = range(-10,10,1)
Ik = np.zeros(len(k))

for i in range(0,len(x)):
    for j in range(0, len(k)):
        if x[i] <= k[j]+1 and x[i]>k[j]:
            Ik[j]+=1
```

By using suitable library functions, here `plt.hist()`, the program code for a histogram is reduced to one line in the code 3.23.

**Listing 3.23** Calculating a histogram using Matplotlib

```
plt.hist(x, bins=20)
```

It leads to the same goal and is less laborious. With the parameter `bins` we determine in both cases how many intervals we want to use and ultimately, how finely the histogram resolves the different values of the variable.

Histograms are extremely effective in reducing the dimension of the data. In the above example, a histogram maps 1000 data values to an interval of 21 numbers. The histogram primarily captures the stochastic properties of the variable.

### *3.7.2 Identification of the Probability Distribution Using Kernel Density Estimators*

Now that we know how to create histograms, we should return to a recurring theme, namely probability. It is important to understand that histograms represent how likely a certain value of a variable is. From our histogram example, we infer that the $x = 0$ occurs particularly frequently, and is therefore more likely than other values. Numbers greater than 100 are completely unlikely.

The **histogram** $\mathcal{H}(x; b)$ is a quantitative, discontinuous, and non-normalized representation of the **probability density** $p(x; \mu, \sigma, \nu, \kappa)$ of a stochastic process $x$.

In inductive statistics, one systematically investigates the gain of information from samples. From here come methods such as the $\chi^2$ test or the Anderson-Darling test, with which we can calculate how well our data is described by a certain distribution.

Although the histogram can be very helpful, it has a crucial disadvantage: it is discontinuous. The division into intervals segments the histogram into fixed blocks. However, the technical process that produced the data and is thus responsible for the histogram does indeed have a continuous probability density.

At this point, the Kernel Density Estimator helps us. The term kernel is used in the field of statistics in various contexts. We will consider it here as the distribution function centered around zero.

A **Kernel Density Estimator** is a continuous estimate of an unknown distribution. It approximates the unknown distribution by a sum of known distributions.

Let's consider an example with synthetic data in Listing 3.24:

**Listing 3.24** Synthetic data for a kernel density estimation

```
import matplotlib.pyplot as plt
import numpy as np

d1 = np.random.normal(loc=70, scale=15, size=300)
d2 = np.random.normal(loc=20, scale=5, size=700)
data = np.hstack([d1,d2])
data = data.reshape((len(data), 1))

plt.hist(data,width=3, bins=50)
plt.show()
```

In this listing, we have drawn events from two Gaussian distributions and histogrammed them. The resulting histogram is shown in blue in Fig. 3.11. The kernel

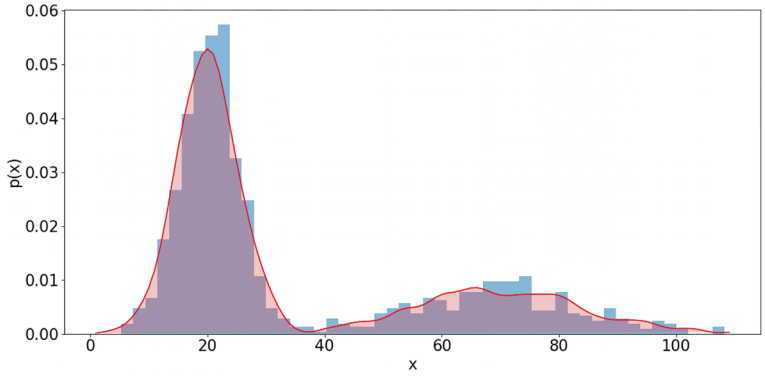


**Fig. 3.11** Scaled histogram (blue) of the input data and overlay of the continuous approximation for $p(x)$ (red) using a kernel density estimator

density estimator uses this input data and approximates a continuous combination of distribution functions. In the following code in Listing 3.25, we assume a Gaussian kernel.

**Listing 3.25** Kernel density estimation using Scikit-Learn

```
from sklearn.neighbors import KernelDensity
model = KernelDensity(bandwidth=3, kernel='gaussian')
model.fit(data)
```

The visualization of the result is fully executed in Listing 3.26 and shows the code that generated the red distribution and the histogram in Fig. 3.11.

**Listing 3.26** Visualization of the kernel density estimation results

```
x = np.asarray([i for i in range(1, 110)])
x = x.reshape((len(x), 1))
y = np.exp(model.score_samples(x))
plt.hist(data, bins=50, density=True, alpha=0.5)
plt.fill(x[:], y, 'r', alpha=0.2)
plt.plot(x[:], y, 'r', alpha=0.9)
plt.tick_params('both', labelsize=20)
plt.xlabel('x', fontsize=20)
plt.ylabel('p(x)', fontsize=20)
plt.show()
```

In line 3 of this listing, an exponential function is used and applied to the random draw from the kernel density model. This is necessary because the function `fit` performs the approximation with logarithmic variants of the distribution.

The quantification of uncertainties is a distinct field of study and is important for the areas of multiparticle physics and the dynamics of complex systems. Here, more extensive approaches are used. For further reading, we recommend the works of P. J. van Leeuwen et al., who use particle filters [4, 6] in the context of meteorological and oceanographic models, as well as the ensemble Kalman assimilation approaches, as discussed by M. Jardak and O. Talagrand in [3].

## 3.8 Determining Key Figures from Data

### *3.8.1 Statistical Key Figures*

One of the most commonly used concepts for processing large amounts of data is the reduction of dimensions to statistical key figures. Here, the aim is to capture a high-dimensional data set with a few characteristic values. Expected value and moments are suitable measures in many cases to effectively reduce data. But they also serve as target values for learned models.

In some process databases, they even go so far as to only store very condensed key figures of time series, e.g., mean, min, and max. This saves the storage of large data sets from the outset. The sensor delivers all values, but only these few key data are retained. Unfortunately, this step also loses information. Especially the dynamics so important for time series, which is mainly expressed in the gradients and oscillations during the data course, cannot be captured by the distribution parameters.

**Example: Acceleration, Deceleration, and Oscillation**
Imagine a motor whose speed you first increase from 0 revolutions per minute upwards, then let it fluctuate between two values, and finally decelerate back to 0 revolutions per minute. For all three scenarios, you can get the same maximum, minimum, and average values and in principle no longer distinguish between the actual states. If you find that most faulty states occur during deceleration, then it is not enough to only capture the statistical key figures. ◀

A machine start-up situation would therefore possibly no longer be distinguishable from the braking situation in terms of data. The example makes it clear that there are processes in which the complete evaluation of a time series is necessary and not just the sole consideration of statistical key figures. This problem is investigated for various scenarios in the steel industry in the works of Arnu et al. [1] and Souza et al. [2].

### *3.8.2 Process-Oriented Key Figures*

In principle, the raw data of a dynamic process should also be fully recorded. So first use the Nyquist criterion to determine the most sensible sampling rate for your data. Since we have already seen that statistical quantities alone are not necessarily suitable for always describing the process well, we would therefore like to give you a recipe to sensibly expand your data collection. For this, we focus on characteristic key points of the actual process and determine them in the following way:

- Determine the statistical characteristics of the time series, but also include the temporal positions at which the minimum, maximum, and mean are reached.
- Try to determine the most important frequencies via the FFT and store them. If this is not possible, reduce the frequency space to 4 to 5 frequency ranges and store the averages in these frequency bands.
- Define characteristic points in your data. In time series, always $(t_i, \boldsymbol{x}_i)$. These points are individual and problem-related, but they can be an enormously important tool for formulating target variables. These points can also be

maxima, minima, or turning points in the time series. It is important to jointly capture $t$ and $x_i$.
- Determine relationships between the characteristic points that are meaningful for your process:
  - Is there a characteristic difference between the maximum and minimum?
  - Do the maximum and minimum have a special relationship to each other?
  - Is the distance of the temporal position of the maximum from the start point of the time series relevant?
- Where are the extreme points of the slope reached? Use the zeros of the derivative to find these points and also store the values $t_i$ and $\frac{dx_i}{dt}(t_i)$.

We want to briefly demonstrate this using our example of the motor current:

**Example: Process-oriented Key Figures of the Motor Current Example**
Consider the course of the data in Fig. 3.2. We use the variable $x(t)$ to describe the course of a curve. The following points are extracted here:

1. The position $x(t = 0)$ is a process-relevant offset.
2. The slope at the point $x(t = 10)$ must be as identical as possible in all curves.
3. The position $t_{\text{Max}}$ must be synchronous for all curves. The maximum is therefore relevant, both in its height and in its temporal position.
4. The position and height of the minimum of each curve are also characteristic and different for each curve.
5. The FFT frequency bands are characteristic. With 5 intervals in the frequency space, the disturbance frequency of the anomaly is also captured.
6. The end position of the curves can also be included as a point. ◀

The consideration of process-oriented key figures is therefore helpful. In the present example, these would be about 10 values and they would be completely sufficient to capture the entire problem.

**Summary**
Before we can deal with specific learning methods, we had to create the basics in this chapter to prepare data specifically for so-called training. The work involved in preparing the data, especially in cleaning and considering various normalizations, should not be underestimated in practice.

We have seen the step of triggering, which is necessary for repetitive problems. Then we considered various transformations that can change our view of the data and help learning methods to train faster and more robustly. With the transformations, aspects of the data became visible that were sometimes not or only difficult to see in the original data.

Building on the mathematical foundations of the previous chapter, we discussed histograms and kernel density estimators as tools to extract probability distributions from data. They close the circle between theoretical ideas and practical evaluation.

The preparation steps have one goal, namely to sharpen the view of the essentials in the data. This is achieved by omitting the irrelevant or by focusing on those properties that contain the relevant information. Fourier transformation, wavelet transformation or mapping to distributions are capable of reducing complex data to a few characteristic numerical values.

We will later get to know the unsupervised learning methods in Chap. 5. Here, the principal component analysis (PCA) and the autoencoder are well-known tools for effectively reducing the dimension of data. They represent, like the methods of this chapter, procedures that are also suitable for preparation.

Chap. 6 and 7 will finally pick up the preprocessing again under the aspect of a possibly automatic process chain. If a digital system independently makes the decision which preprocessing is best suited, we finally find ourselves in another area of artificial intelligence: knowledge available for the machine.

## Tasks

**3.1** How can you generate NaNs in a dataset yourself?

**3.2** Program the interpolation from equation (3.1)!

**3.3** Determine the convolution of the functions $f \circ g$ with

$$f(x) = \operatorname{sech}(x) \tag{3.18}$$

and

$$g(x) = 2 * \delta(x - 10) + 7 * \delta(x - 80). \tag{3.19}$$

Present the result graphically! Here, the $\delta$ function is used, which is defined over

$$\delta(x - a) = \begin{cases} 1 \text{ für } x = a \\ 0 \text{ sonst,} \end{cases} \tag{3.20}$$

.

**3.4** Approach the result of task 3.2 with the kernel density estimator. Use kernels that we have not explicitly addressed in the text: a) Cauchy kernel, b) Epanechnikov kernel, and c) Picard kernel. What differences do you notice?

**3.5** What are the process-oriented, characteristic features of the triggered example in Fig. 3.6?

## References

1. D. Arnu, E. Yaqub, C. Mocci, V. Colla, M. J. Neuer, G. Fricout, X. Renard, C. Mozzati, and P. Gallinari, *A reference architecture for quality improvement in steel production*. Springer, 2017.
2. A. d. M. Souza, D. Arnu, F. Temme, E. Klapic, R. Klinkenberg, M. J. Neuer, X. Renard, P. Gallinari, C. Mozzati, C. Mocci, and G. Fricout, „Data mining and modelling," *Steel Times International*, 2018.
3. M. Jardak and O. Talagrand, „Ensemble variational assimilation as a probabilistic estimator – part 1: The linear and weak non-linear case," *Nonlinear Processes in Geophysics*, vol. 25, no. 3, pp. 565–587, 2018. [Online]. Available: https://npg.copernicus.org/articles/25/565/2018/
4. P. J. V. Leeuven, „Aspects of particle filtering in high dimensional spaces," *Lecture Notes in Computer Science*, vol. 8964, pp. 251–262, 2015.
5. M. J. Neuer, A. Quick, T. George, and N. Link, „Anomaly and causality analysis in process data streams using machine learning with specialized eigenspace topologies," in *Proceedings of ESTAD 2019*, 2019.
6. S. Pathiraja and P. J. van Leeuwen, „Multiplicative non-gaussian model error estimation in data assimilation," 2021.

# Chapter 4
# Supervised Learning


**Keywords** Supervised learning · Adaptive filters · Evolutionary algorithms · Neural networks · Decision trees · Gradient descent methods

This chapter discusses concepts of supervised learning. It introduces a general approach to learning methods, which serves as a basis for the following approaches. Starting with a naive, evolutionary algorithm, we discuss the Least-Mean-Squares (LMS) adaptive filter, neural networks, recurrent neural networks, and decision trees in the course of the chapter. Example implementations are provided for each method, giving the reader a starting point for their own projects.

Fig. 4.1 provides an overview of how the following chapters benefit from the content presented here. A central concept is the cost function $J$. It allows us to formulate learning as an optimization problem. This basic approach will accompany us in Chap. 4 and 5.

The learning methods themselves form the basis for Chap. 6. Chapter 7 shows methods that are applied to a trained model to better understand it.

## 4.1 Learning

### *4.1.1 What does Learning Actually Mean?*

- **Associative Learning.** In associative learning, the aim is to link two or more events together in the learner. Mathematically, if-then links are to be developed, where

$$X \rightarrow Y \rightarrow Z \tag{4.1}$$



M. J. Neuer, *Machine Learning for Engineers*,
https://doi.org/10.1007/978-3-662-69995-9_4